

PSPO line-segment implementation

Implementation guide | Code inspected on 7 October 2026

PSPO keeps a last safe actor checkpoint and obtains a proposed actor from PPO. The line-segment variant does not learn a box around the checkpoint. It searches for a certified prefix of the straight path towards that proposal, then keeps the far endpoint of the prefix. Safety labels come from the shield/certificate dataset; no reward values are read by the segment search. Rewards can influence the PPO proposal, not the certification predicate.

1. Fix the direction; search only the step length

Let θ_k denote the last safe actor, and θ_p the proposed actor. Actor weights and biases are flattened in `model.parameters()` order. The critic is not part of this certified parameter vector.

$$\Delta = \theta_p - \theta_k, \quad \theta(t) = \theta_k + t\Delta, \quad 0 \leq t \leq 1.$$

The desired region is a prefix, not an arbitrary disconnected collection of safe points:

$$\mathcal{L}_\alpha = \{\theta_k + t\Delta : 0 \leq t \leq \alpha\}, \quad 0 < \alpha \leq 1.$$



Schematic in parameter space; not a measured step length.

2. Represent each subsegment exactly

For any interval $[a,b]$ of step lengths, the implementation uses a rank-one zonotope: a centre c and one generator g , with a single uncertain scalar z .

$$c = \theta_k + \frac{a+b}{2}\Delta, \quad g = \frac{b-a}{2}\Delta, \quad \theta = c + zg, \quad z \in [-1, 1].$$

This is exactly the same parameter set as $\theta_k + t\Delta$ for t in $[a,b]$. Crucially, the same z controls every weight and bias, across every layer. Parameters are not allowed to vary independently as they would inside an enclosing orthotope. The geometry is exact; the neural-network output bounds used later may still be conservative.

3. Return one region even though certification is split

Once a positive α is accepted, the returned `ZonotopeRegion` has centre $\theta_k + \alpha\Delta/2$, generator $\alpha\Delta/2$ and coefficient bounds $[-1,1]$. Its union of certified subsegments is the complete prefix. The search restores the scratch/frozen actor to θ_k in a finally block, including on early return; accepting or projecting the live actor is the caller's job.

How the whole segment is certified

The test covers a continuum of policies, not just a grid of checkpoints.

4. Bound the logits for all policies in a subsegment

`certify_sub_segment` copies the subsegment centre into a frozen actor and calls `bound_forward_pass` with `use_zonotopes=True`, the rank-one generator, and `z` in $[-1,1]$. Shared-coefficient zonotope/interval arithmetic propagates through Linear, Tanh or ReLU layers, with conservative bounds for nonlinear and bilinear terms. The final result is an interval for each action logit.

$$L_j(s; a, b) \leq f_j(s; \theta_k + t\Delta) \leq U_j(s; a, b) \quad \text{for all } t \in [a, b].$$

Point-state datasets provide (state, safe-action mask). Input-box datasets instead provide (lower state, upper state, safe-action mask), so the same test also ranges over the represented input box. Safe-action masks are multi-hot labels from the safety certificate, not reward targets.

5. Require a safe-vs-unsafe logit separation

For a state s , let $A_{\text{safe}}(s)$ be its labelled safe actions and $A_{\text{unsafe}}(s)$ the remaining actions. The default `multi_label_mode='all'` requires every safe action's lower bound to exceed every unsafe action's upper bound:

$$\min_{j \in A_{\text{safe}}(s)} L_j(s; a, b) > \max_{j \in A_{\text{unsafe}}(s)} U_j(s; a, b).$$

This is stronger than merely requiring the greedy action to be safe. With `mode='any'`, the left-hand min becomes max: one safe action's lower bound must beat all unsafe upper bounds. Both tests use a strict inequality, so a tie does not certify. A state with no unsafe actions passes trivially; a state with no safe actions fails.

The hard accuracy is aggregated with 'min'. Therefore every represented state in every batch must pass; this is not a mean-safety threshold. All dataset batches are materialised once and reused. The segment engine checks the full supplied certificate dataset, irrespective of the orthotope engine's certificate-samples setting.

6. Split a prefix into K pieces

For a candidate α , `certify_segment` divides $[0, \alpha]$ into K equal intervals and verifies each one:

$$C_K(\alpha) = \bigwedge_{i=0}^{K-1} C\left(\frac{i\alpha}{K}, \frac{(i+1)\alpha}{K}\right).$$

Here $C(a,b)$ means that every dataset batch passes the preceding bound test. Since the pieces cover $[0, \alpha]$, a passing $C_K(\alpha)$ certifies every policy in that prefix, under the verifier's bound-propagation assumptions. Either a failing state batch or a failing subsegment stops that test immediately. Splitting aims to reduce bound conservatism; no intermediate policy is certified by sampling alone.

The implemented search

Defaults: tolerance $\varepsilon = 0.001$, initial $K = 4$, maximum $K = 8$.

7. Full-step test, then bisection

The following is the actual control flow of `compute_segment_rashomon_set`, omitting tensor handling and diagnostic counters:

```
K = initial_splits
repeat:
    if C_K(1): return certified prefix [0, 1]
    lo, hi = 0, 1
    while hi - lo > tolerance:
        mid = (lo + hi) / 2
        if C_K(mid): lo = mid
        else: hi = mid
    if lo > 0 or K >= max_splits: break
    K = min(2 * K, max_splits)
if lo > 0: return certified prefix [0, lo]
else: return no region (caller reverts to theta_k)
always restore the frozen actor to theta_k
```

The lower endpoint is advanced only after a passing certificate. At $\varepsilon = 0.001$, a failed full-step test is followed by 10 midpoint tests, since $2^{-10} < \varepsilon$. Refinement from $K = 4$ to $K = 8$ happens ONLY when no positive prefix was found at $K = 4$. A positive $K = 4$ result is returned immediately; the code does not then try $K = 8$ to improve it. At each refinement level, $\alpha = 1$ is tested again first.

8. What the tolerance does and does not mean

The bisection aims to find a long certifiable prefix. Its tolerance controls the final search bracket in α . A failed test means 'not certified', not necessarily 'unsafe'. The accepted step need not be within ε of the longest truly safe step: the verifier can be conservative. Moreover, a maximality claim for bisection needs a monotonicity justification for the implemented bound propagation and re-partitioning; the loop itself only ensures that a returned positive lower endpoint passed its certificate. It does not exhaustively test every possible α .

9. A concrete example, checked against the code

Take a one-layer actor with two actions, zero weights and initial biases (0,5). Only action 1 is labelled safe. The PPO proposal changes the biases to (10,5); along the path the unsafe logit is $10t$ and the safe logit remains 5.

$$\text{Certificate condition : } 5 > 10\alpha, \quad \text{so } \alpha < 0.5.$$

The full step fails, and the first midpoint 0.5 fails because the logits tie. With the defaults, the code returns $\alpha = 0.4990234375$, $K = 4$, after 11 prefix-certificate evaluations. The accepted biases are (4.990234375,5), so action 1 remains the unique greedy winner. The frozen actor is restored to (0,5). This is an illustrative unit-scale example, not an environment experiment result.

Applying the result and interpreting safety

10. The accepted actor is the prefix endpoint

For the returned region, put $c = \theta_k + \alpha\Delta/2$ and $g = \alpha\Delta/2$. `project_flat_to_zonotope` uses the exact rank-one Euclidean projection, not an iterative optimisation:

$$z^* = \text{clip}\left(\frac{(\theta_p - c)^T g}{g^T g}, -1, 1\right), \quad \theta_{\text{new}} = c + z^* g.$$

For a nonzero proposal direction and $0 < \alpha \leq 1$, the unclipped coefficient is $2/\alpha - 1 \geq 1$. Thus $z^* = 1$ and the accepted actor is exactly $\theta_k + \alpha\Delta$. If $\alpha = 1$, the whole proposal is accepted. If no positive prefix certifies, the live actor reverts to θ_k . A zero generator is handled by returning the centre. The accepted actor becomes the next last-safe checkpoint.

11. Segment-first versus verify-first + segment

Segment-first (AdaptiveSafePPOV2) searches a certified prefix at the safety-enforcement checkpoint, then accepts its endpoint or reverts. Its optional candidate audit is diagnostic, not the verify-first bypass.

Verify-first (AdaptiveSafePPO, including the newly launched combined variant) first verifies the proposed actor on the certificate set, using the configured 'all' or 'any' invariant. If it passes, it is accepted directly and no segment is computed. Otherwise the same segment search and projection described above run. Hence a direct verify-first acceptance certifies the endpoint, not necessarily every policy on the path from the previous checkpoint to it.

12. Scope and cost

The certificate covers greedy actions on the supplied states or input boxes, relative to their labelled safe actions. It does not establish reward improvement or goal success, and does not certify stochastic softmax sampling, which can select unsafe actions. Trajectory safety additionally needs appropriate shield labels and state coverage.

Enforcement applies at configured checkpoints and finalisation, not after every unconstrained gradient step. An old segment is not reused to restrict the next proposal: its direction changes. The segment engine needs no gradients, surrogate optimisation or temperature calibration. Each prefix test costs at most K dataset passes, with early exits. A failed-full-step search uses 11 tests; an unsuccessful $K = 4$ search followed by $K = 8$ uses at most 22. Diagnostics count these tests as `iterations_run`, not gradient iterations.

13. Where to read the implementation

Paths below are relative to the repository root; line numbers reflect the inspection date.

```
core/src/segment_rashomon.py
:104 certify_sub_segment; :152 certify_segment
:184 compute_segment_rashomon_set; :346 segment_endpoint
core/src/verification/verify.py
:35 bound_forward_pass; :187 bound_multi_label_accuracy
core/provably_safe_policy_optimisation/regions.py
:182 project_flat_to_zonotope
core/provably_safe_policy_optimisation/adaptive_safe_ppo.py
:1305 segment direction/call; :1401 verify-first enforcement
core/provably_safe_policy_optimisation/adaptive_safe_ppo_v2.py
:702 segment-first train-phase enforcement
```